Qolqe Wallet

Pruebas de recargas — con Karate

Explicación de los 71 casos de api.feature

Documento para exposición y consulta durante una demostración en vivo.

71 casos	64 aprobados	7 fallidos

Resultados de la validación documentada del 17 de octubre de 2026. Los siete fallos reproducen tres defectos conocidos; no se volvió a ejecutar la suite para producir este PDF.

Fuente: docs/casos-api-recargas.md. El cuerpo del documento conserva sus 71 explicaciones en el mismo orden. La siguiente página agrega un guion sugerido para la demo.

Guion sugerido para la exposición

1. Explica el objetivo: probar por HTTP las historias de recargas de una API Python con Karate, un ejecutor basado en Java.

2. Muestra docs/requisitos.md y relaciona una regla con su escenario en api.feature. Explica Given, When, Then y las aserciones match.

3. Muestra la preparación: datos restaurados, login, token y consulta de saldo. Explica por qué la sesión debe estar activa.

4. Ejecuta un caso que pasa: saldo exacto de S/ 5. Luego demuestra el bono de S/ 30, que falla por un defecto del producto.

5. Abre el reporte HTML y compara el valor esperado con el obtenido. Un escenario negativo aprobado es un rechazo correcto; un escenario rojo no implica que Karate esté mal instalado.

6. Cierra mostrando la cobertura de las siete historias y los tres defectos registrados. La visualización del bono antes de confirmar requiere una prueba de interfaz separada.

Comandos para la demo

```
source env/bin/activate

python scripts/karate_local.py --tags @recargas --name "Saldo exacto"

python scripts/karate_local.py --tags @R-G1-02

python scripts/karate_local.py --tags @recargas
```

El ejecutor inicia y apaga su propia API con base temporal. No necesitas levantar uvicorn manualmente. La primera ejecución requiere descargar Karate si no está en .tools/. Cada comando actualiza el reporte: tras demostrar los casos individuales, ejecuta la suite completa si quieres mostrar el resumen de los 71 casos.

Explicación caso por caso de api.feature

Este documento sigue el orden de api.feature (../api.feature). Explica los **71 casos ejecutables**: cada fila de un `Scenario Outline` cuenta como un caso independiente. Los enlaces de cada entrada apuntan al escenario de origen.

Para instalar, entender la sintaxis y consultar la matriz por historia, lee también la guía de Karate (guia-karate-recargas.md). Los requisitos son la fuente de los resultados esperados; los resultados actuales no sustituyen esos requisitos.

Preparación común a todos los casos

Antes de **cada caso**, incluso cada fila de Examples, se restablece el reloj, se restauran los datos, se inicia sesión como Ana (`987654321`, PIN `4826`) y se configura el token. Una consulta autenticada debe confirmar saldo inicial `1500.00`. Ana tiene una programada activa y una pausada. Los casos que comparan saldo durante el proceso diario aíslan las programadas y el débito de servicios de la semilla. Cuando cambian la fecha, vuelven a obtener un token para evitar sesiones vencidas.

Las variables `id`, `fecha`, `antes` o `historial` guardan datos de respuestas para las siguientes solicitudes del mismo escenario. No son datos compartidos con la prueba anterior. Los importes devueltos por la API se comparan como texto decimal.

Cómo interpretar el resultado indicado

El resultado indicado en cada entrada corresponde a la validación documentada del **17 de octubre de 2026**: 64 aprobados y 7 fallidos. No se volvió a ejecutar la aplicación para crear este documento. Un caso negativo aprobado significa que la API rechazó la operación como se esperaba. Tras la primera aserción fallida, Karate no ejecuta las restantes comprobaciones de ese escenario.

Casos, en el orden del archivo

Caso 01. Recargar S/ 10 y descontar el saldo

Origen: api.feature, línea 30 (../api.feature#L30). **Etiquetas:** `@HU-G1-01`.

Envía una recarga manual de S/ 10 al 987111222. Espera 201, código de operación textual, origen MANUAL y monto 10.00. Comprueba saldo 1490.00 tanto en el comprobante como en una consulta independiente. Detecta descuentos incorrectos y comprobantes inconsistentes.

Resultado documentado: aprobado.

Caso 02. Recarga exitosa a un número válido por S/ 5

Origen: api.feature, línea 52 (../api.feature#L52). **Etiquetas:** `@HU-G1-01`, `@R-G1-01`, `@HU-G1-01-CA1-VALID`.

Envía monto 5 al celular 999999999. Espera 201, origen MANUAL y código textual. El comprobante debe devolver monto `5.00`. Esta fila verifica la aceptación de S/ 5; el bono se comprueba por separado, de modo que un defecto de bono no impida validar la lista de montos permitidos.

Resultado documentado: aprobado.

Caso 03. Recarga exitosa a un número válido por S/ 10

Origen: api.feature, línea 52 (../api.feature#L52). **Etiquetas:** `@HU-G1-01`, `@R-G1-01`, `@HU-G1-01-CA1-VALID`.

Envía monto 10 al celular 999999999. Espera 201, origen MANUAL y código textual. El comprobante debe devolver monto `10.00`. Esta fila verifica la aceptación de S/ 10; el bono se comprueba por separado, de modo que un defecto de bono no impida validar la lista de montos permitidos.

Resultado documentado: aprobado.

Caso 04. Recarga exitosa a un número válido por S/ 20

Origen: api.feature, línea 52 (../api.feature#L52). **Etiquetas:** `@HU-G1-01`, `@R-G1-01`, `@HU-G1-01-CA1-VALID`.

Envía monto 20 al celular 999999999. Espera 201, origen MANUAL y código textual. El comprobante debe devolver monto `20.00`. Esta fila verifica la aceptación de S/ 20; el bono se comprueba por separado, de modo que un defecto de bono no impida validar la lista de montos permitidos.

Resultado documentado: aprobado.

Caso 05. Recarga exitosa a un número válido por S/ 30

Origen: api.feature, línea 52 (../api.feature#L52). **Etiquetas:** `@HU-G1-01`, `@R-G1-01`, `@HU-G1-01-CA1-VALID`.

Envía monto 30 al celular 999999999. Espera 201, origen MANUAL y código textual. El comprobante debe devolver monto `30.00`. Esta fila verifica la aceptación de S/ 30; el bono se comprueba por separado, de modo que un defecto de bono no impida validar la lista de montos permitidos.

Resultado documentado: aprobado.

Caso 06. Recarga exitosa a un número válido por S/ 50

Origen: api.feature, línea 52 (../api.feature#L52). **Etiquetas:** `@HU-G1-01`, `@R-G1-01`, `@HU-G1-01-CA1-VALID`.

Envía monto 50 al celular 999999999. Espera 201, origen MANUAL y código textual. El comprobante debe devolver monto `50.00`. Esta fila verifica la aceptación de S/ 50; el bono se comprueba por separado, de modo que un defecto de bono no impida validar la lista de montos permitidos.

Resultado documentado: aprobado.

Caso 07. Recarga exitosa a un número válido por S/ 100

Origen: api.feature, línea 52 (../api.feature#L52). **Etiquetas:** `@HU-G1-01`, `@R-G1-01`, `@HU-G1-01-CA1-VALID`.

Envía monto 100 al celular 999999999. Espera 201, origen MANUAL y código textual. El comprobante debe devolver monto `100.00`. Esta fila verifica la aceptación de S/ 100; el bono se comprueba por separado, de modo que un defecto de bono no impida validar la lista de montos permitidos.

Resultado documentado: aprobado.

Caso 08. Rechazar celular 899999999 y monto 50

Origen: api.feature, línea 70 (../api.feature#L70). **Etiquetas:** `@HU-G1-01`, `@R-G1-01`, `@HU-G1-01-CA1-ERROR`.

Envía celular 899999999 y monto `50`: el celular no empieza con 9. Espera 422 con código `CELULAR_INVALIDO` y un mensaje textual. Después consulta saldo y exige 1500.00: el intento inválido no debe descontar dinero. El texto cinco falla validación de formato (SOLICITUD_INVALIDA), antes de

evaluar las reglas de montos.

Resultado documentado: aprobado.

Caso 09. Rechazar celular 99999999 y monto 50

Origen: api.feature, línea 70 (../api.feature#L70). **Etiquetas:** `@HU-G1-01`, `@R-G1-01`, `@HU-G1-01-CA1-ERROR`.

Envía celular 99999999 y monto `50`: el celular tiene ocho dígitos. Espera 422 con código `CELULAR_INVALIDO` y un mensaje textual. Después consulta saldo y exige 1500.00: el intento inválido no debe descontar dinero. El texto cinco falla validación de formato (SOLICITUD_INVALIDA), antes de evaluar las reglas de montos.

Resultado documentado: aprobado.

Caso 10. Rechazar celular 999999999 y monto 0

Origen: api.feature, línea 70 (../api.feature#L70). **Etiquetas:** `@HU-G1-01`, `@R-G1-01`, `@HU-G1-01-CA1-ERROR`.

Envía celular 999999999 y monto `0`: el monto no pertenece a la lista de importes permitidos. Espera 422 con código `MONTO_NO_PERMITIDO` y un mensaje textual. Después consulta saldo y exige 1500.00: el intento inválido no debe descontar dinero. El texto cinco falla validación de formato (SOLICITUD_INVALIDA), antes de evaluar las reglas de montos.

Resultado documentado: aprobado.

Caso 11. Rechazar celular 999999999 y monto 15

Origen: api.feature, línea 70 (../api.feature#L70). **Etiquetas:** `@HU-G1-01`, `@R-G1-01`, `@HU-G1-01-CA1-ERROR`.

Envía celular 999999999 y monto `15`: el monto no pertenece a la lista de importes permitidos. Espera 422 con código `MONTO_NO_PERMITIDO` y un mensaje textual. Después consulta saldo y exige 1500.00: el intento inválido no debe descontar dinero. El texto cinco falla validación de formato (SOLICITUD_INVALIDA), antes de evaluar las reglas de montos.

Resultado documentado: aprobado.

Caso 12. Rechazar celular 999999999 y monto -15

Origen: api.feature, línea 70 (../api.feature#L70). **Etiquetas:** `@HU-G1-01`, `@R-G1-01`, `@HU-G1-01-CA1-ERROR`.

Envía celular 999999999 y monto `-15`: el monto no pertenece a la lista de importes permitidos. Espera 422 con código `MONTO_NO_PERMITIDO` y un mensaje textual. Después consulta saldo y exige 1500.00: el intento inválido no debe descontar dinero. El texto cinco falla validación de formato (SOLICITUD_INVALIDA), antes de evaluar las reglas de montos.

Resultado documentado: aprobado.

Caso 13. Rechazar celular 999999999 y monto 50.5

Origen: api.feature, línea 70 (../api.feature#L70). **Etiquetas:** `@HU-G1-01`, `@R-G1-01`, `@HU-G1-01-CA1-ERROR`.

Envía celular 999999999 y monto `50.5`: el monto no pertenece a la lista de importes permitidos. Espera 422 con código `MONTO_NO_PERMITIDO` y un mensaje textual. Después consulta saldo y exige 1500.00: el intento inválido no debe descontar dinero. El texto cinco falla validación de formato (SOLICITUD_INVALIDA), antes de evaluar las reglas de montos.

Resultado documentado: aprobado.

Caso 14. Rechazar celular 999999999 y monto cinco

Origen: api.feature, línea 70 (../api.feature#L70). **Etiquetas:** `@HU-G1-01`, `@R-G1-01`, `@HU-G1-01-CA1-ERROR`.

Envía celular 999999999 y monto `cinco`: el monto es texto no convertible a decimal. Espera 422 con código `SOLICITUD_INVALIDA` y un mensaje textual. Después consulta saldo y exige 1500.00: el intento inválido no debe descontar dinero. El texto cinco falla validación de formato (SOLICITUD_INVALIDA), antes de evaluar las reglas de montos.

Resultado documentado: aprobado.

Caso 15. Validar el monto descontado de la cuenta, el bono aplicado y el SMS

Origen: api.feature, línea 95 (../api.feature#L95). **Etiquetas:** `@HU-G1-01`, `@HU-G1-01-CA2`.

Recarga S/ 50 al 987111222. Espera bono 2.50 y saldo 1450.00, confirmado mediante GET del saldo. Consulta los SMS del destinatario: debe existir uno y mencionar monto 50.00 y bono 2.50. No basta con que la operación responda 201.

Resultado documentado: aprobado.

Caso 16. Bono desde S/ 30 con tope de S/ 4 para monto 5

Origen: api.feature, línea 117 (../api.feature#L117). **Etiquetas:** `@HU-G1-01`, `@R-G1-02`.

Recarga S/ 5 al 999999999 y espera 201 con bono `0.00`. No alcanza el umbral de S/ 30: no corresponde bono.

Resultado documentado: aprobado.

Caso 17. Bono desde S/ 30 con tope de S/ 4 para monto 10

Origen: api.feature, línea 117 (../api.feature#L117). **Etiquetas:** `@HU-G1-01`, `@R-G1-02`.

Recarga S/ 10 al 999999999 y espera 201 con bono `0.00`. No alcanza el umbral de S/ 30: no corresponde bono.

Resultado documentado: aprobado.

Caso 18. Bono desde S/ 30 con tope de S/ 4 para monto 20

Origen: api.feature, línea 117 (../api.feature#L117). **Etiquetas:** `@HU-G1-01`, `@R-G1-02`.

Recarga S/ 20 al 999999999 y espera 201 con bono `0.00`. No alcanza el umbral de S/ 30: no corresponde bono.

Resultado documentado: aprobado.

Caso 19. Bono desde S/ 30 con tope de S/ 4 para monto 30

Origen: api.feature, línea 117 (../api.feature#L117). **Etiquetas:** `@HU-G1-01`, `@R-G1-02`.

Recarga S/ 30 al 999999999 y espera 201 con bono `1.50`. Es exactamente el umbral: desde 30 incluye el propio 30.

Resultado documentado: fallido. DEF-03: devuelve bono 0.00 en lugar de 1.50. Los pasos posteriores al fallo no llegaron a verificarse.

Caso 20. Bono desde S/ 30 con tope de S/ 4 para monto 50

Origen: api.feature, línea 117 (../api.feature#L117). **Etiquetas:** `@HU-G1-01`, `@R-G1-02`.

Recarga S/ 50 al 999999999 y espera 201 con bono `2.50`. El 5 % de 50 es 2.50, por debajo del tope.

Resultado documentado: aprobado.

Caso 21. Bono desde S/ 30 con tope de S/ 4 para monto 100

Origen: api.feature, línea 117 (../api.feature#L117). **Etiquetas:** `@HU-G1-01`, `@R-G1-02`.

Recarga S/ 100 al 999999999 y espera 201 con bono `4.00`. El 5 % de 100 sería 5.00, pero el tope obliga a devolver 4.00.

Resultado documentado: aprobado.

Caso 22. Rechazar la sexta recarga manual sin descontar saldo

Origen: api.feature, línea 134 (../api.feature#L134). **Etiquetas:** `@HU-G1-01`, `@R-G1-03`.

Realiza cinco recargas de S/ 5 al mismo número, cada una con 201. La sexta debe responder 422 LIMITE_RECARGAS. Después consulta el saldo: debe ser 1475.00, sin cobrar la operación rechazada. Todas las solicitudes pertenecen al mismo caso, sin reiniciar datos entre ellas.

Resultado documentado: fallido. DEF-04: se permite una sexta recarga; responde 201 o descuenta dinero cuando debía rechazarla. Los pasos posteriores al fallo no llegaron a verificarse.

Caso 23. Una recarga programada cuenta junto a cuatro manuales

Origen: api.feature, línea 166 (../api.feature#L166). **Etiquetas:** `@HU-G1-01`, `@R-G1-03`.

Crea una programada de S/ 5 para el 999999999. Avanza a su próxima fecha y vuelve a iniciar sesión. Ejecuta el proceso diario y confirma una recarga de origen PROGRAMADA para ese número. Luego acepta cuatro manuales de S/ 5 y exige que la siguiente se rechace con 422 LIMITE_RECARGAS. Verifica que ambos orígenes comparten el contador diario.

Resultado documentado: fallido. DEF-04: se permite una sexta recarga; responde 201 o descuenta dinero cuando debía rechazarla. Los pasos posteriores al fallo no llegaron a verificarse.

Caso 24. Permitir recargar al mismo número al día siguiente

Origen: api.feature, línea 213 (../api.feature#L213). **Etiquetas:** `@HU-G1-01`, `@R-G1-03`.

Fija el reloj a las 23:50 del 15 de octubre de 2026. Inicia sesión y hace cinco recargas al mismo número. Avanza once minutos, hasta las 00:01 del día siguiente, y vuelve a iniciar sesión. La siguiente recarga debe responder 201: el límite depende del día calendario de Lima, no de una ventana móvil de 24 horas.

Resultado documentado: aprobado.

Caso 25. Aceptar tres activas y rechazar la cuarta sin contar las pausadas

Origen: api.feature, línea 262 (../api.feature#L262). **Etiquetas:** `@HU-G1-03`, `@R-G1-04`.

Confirma la semilla: una programada ACTIVA y una PAUSADA. Crea dos nuevas, llegando a tres activas, y lo verifica en el listado. Una cuarta creación debe responder 422 LIMITE_PROGRAMADAS. La pausada no consume un cupo.

Resultado documentado: aprobado.

Caso 26. Pausar libera un cupo pero reanudar no permite superar tres activas

Origen: api.feature, línea 294 (../api.feature#L294). **Etiquetas:** `@HU-G1-03`, `@R-G1-04`, `@HU-G1-05`.

Parte de una activa y una pausada, crea dos activas y alcanza tres. Pausa la activa semilla y crea otra aprovechando el cupo liberado. Intenta reanudar la pausada inicial: espera 422 LIMITE_PROGRAMADAS. Si ese rechazo pasa, verifica que siguen existiendo solo tres activas y que la programada rechazada permanece pausada.

Resultado documentado: fallido. DEF-05: reanudar responde 200 y supera las tres activas, en lugar de rechazar con 422. Los pasos posteriores al fallo no llegaron a verificarse.

Caso 27. Validar el día de programación 1

Origen: api.feature, línea 335 (../api.feature#L335). **Etiquetas:** `@HU-G1-03`, `@R-G1-05`.

Intenta programar S/ 10 al 999999999 para el día 1. Es un límite válido del intervalo 1–28: espera 201, día 1 y estado ACTIVA.

Resultado documentado: aprobado.

Caso 28. Validar el día de programación 28

Origen: api.feature, línea 335 (../api.feature#L335). **Etiquetas:** `@HU-G1-03`, `@R-G1-05`.

Intenta programar S/ 10 al 999999999 para el día 28. Es un límite válido del intervalo 1–28: espera 201, día 28 y estado ACTIVA.

Resultado documentado: aprobado.

Caso 29. Validar el día de programación 0

Origen: api.feature, línea 335 (../api.feature#L335). **Etiquetas:** `@HU-G1-03`, `@R-G1-05`.

Intenta programar S/ 10 al 999999999 para el día 0. Está fuera del intervalo 1–28: espera 422 PROGRAMACION_INVALIDA.

Resultado documentado: aprobado.

Caso 30. Validar el día de programación 29

Origen: api.feature, línea 335 (../api.feature#L335). **Etiquetas:** `@HU-G1-03`, `@R-G1-05`.

Intenta programar S/ 10 al 999999999 para el día 29. Está fuera del intervalo 1–28: espera 422 PROGRAMACION_INVALIDA.

Resultado documentado: aprobado.

Caso 31. Rechazar duplicados activos y pausados y permitir recrear tras eliminar

Origen: api.feature, línea 350 (../api.feature#L350). **Etiquetas:** `@HU-G1-03`, `@R-G1-05`.

Crea una programada para un número y día determinados. Repetirlos debe responder 409 PROGRAMACION_DUPLICADA. Pausa la original y repite: debe seguir rechazándose porque no está eliminada. Finalmente la elimina con 204 y recrea la misma combinación con 201. Comprueba que el criterio de duplicidad no depende solo del estado ACTIVA.

Resultado documentado: aprobado.

Caso 32. Permitir mismo celular con otro día y otro celular con el mismo día

Origen: api.feature, línea 379 (../api.feature#L379). **Etiquetas:** `@HU-G1-03`, `@R-G1-05`.

Pausa la activa semilla para disponer de tres cupos. Crea dos programadas al mismo celular en días distintos (12 y 13) y otra a un celular diferente en el día 12. Todas deben responder 201: el duplicado se define por la pareja número/día, no por cada campo de manera aislada.

Resultado documentado: aprobado.

Caso 33. El límite alcanzado por un número no bloquea otro número

Origen: api.feature, línea 408 (../api.feature#L408). **Etiquetas:** `@HU-G1-01`, `@R-G1-03`.

Recarga cinco veces S/ 5 al 999999999 y comprueba que una sexta se rechace. Después recarga S/ 5 al 999999998: debe responder 201 y dejar el saldo en 1470.00. El límite debe afectar al destino que lo alcanzó. Actualmente el caso se detiene en el rechazo de la sexta, por lo que la independencia del segundo número aún no se valida en esa ejecución.

Resultado documentado: fallido. DEF-04: se permite una sexta recarga; responde 201 o descuenta dinero cuando debía rechazarla. Los pasos posteriores al fallo no llegaron a verificarse.

Caso 34. Una programada no puede superar cinco recargas manuales del mismo día

Origen: api.feature, línea 446 (../api.feature#L446). **Etiquetas:** `@HU-G1-01`, `@R-G1-03`, `@HU-G1-07`, `@HU-G1-07-CA3`.

Desactiva el débito automático de servicios y pausa la programada activa semilla para aislar el saldo. Crea una programada de S/ 5, adelanta a su fecha e inicia sesión de nuevo. Antes de procesarla, realiza cinco recargas manuales al mismo número y guarda saldo e historial. Al procesar, no debe haber descuento adicional ni una sexta recarga. Si esas comprobaciones pasan, exige estado FALLIDA, próxima fecha posterior y SMS al titular con el motivo del límite.

Resultado documentado: fallido. DEF-04: se permite una sexta recarga; responde 201 o descuenta dinero cuando debía rechazarla. Los pasos posteriores al fallo no llegaron a verificarse.

Caso 35. Saldo insuficiente no registra recarga ni descuenta dinero

Origen: api.feature, línea 542 (../api.feature#L542). **Etiquetas:** `@HU-G1-01`, `@HU-G1-01-CA3`.

Fija saldo 4.99 y guarda el historial. Intenta recargar S/ 5: espera 422 SALDO_INSUFICIENTE. Consulta saldo e historial otra vez: deben conservarse en 4.99 y sin nuevos registros. Es un caso negativo que

pasa cuando la API rechaza correctamente.

Resultado documentado: aprobado.

Caso 36. Saldo exacto permite recargar y termina en cero

Origen: api.feature, línea 566 (../api.feature#L566). **Etiquetas:** `@HU-G1-01`, `@HU-G1-01-CA3`.

Fija saldo 5.00 y solicita una recarga de S/ 5. Debe aceptarse con 201 y saldo final 0.00, también confirmado con una consulta independiente. Comprueba el borde válido de la regla de saldo, complementando el caso de saldo insuficiente.

Resultado documentado: aprobado.

Caso 37. Configuración publica montos y parámetros del bono según requisitos

Origen: api.feature, línea 582 (../api.feature#L582). **Etiquetas:** `@HU-G1-01`, `@configuracion`.

Consulta GET /api/recargas/configuracion. Exige los montos 5, 10, 20, 30, 50 y 100; mínimo de bono 30, porcentaje 5, tope 4.00 y máximo diario 5. Verifica el contrato publicado por la API. No demuestra que la interfaz muestre correctamente un bono estimado antes de confirmar.

Resultado documentado: aprobado.

Caso 38. Historial devuelve las últimas veinte recargas con campos y orden correctos

Origen: api.feature, línea 591 (../api.feature#L591). **Etiquetas:** `@HU-G1-02`.

Crea 21 recargas de S/ 5 a celulares distintos, avanzando un minuto entre operaciones para distinguir su orden. Guarda los comprobantes y consulta el historial. Debe devolver exactamente las últimas 20, en orden inverso a la creación y con los mismos códigos, celulares, montos, bonos, orígenes y fechas. También comprueba que los 21 códigos sean únicos. Los destinos distintos evitan confundir el límite diario con la prueba del historial.

Resultado documentado: aprobado.

Caso 39. Alias omitido es opcional

Origen: api.feature, línea 792 (../api.feature#L792). **Etiquetas:** `@HU-G1-03`, `@HU-G1-03-CA1`.

Crea una programada sin enviar alias. Espera 201 y acepta que alias sea nulo o que el servidor omita el campo. Esto valida que el alias es opcional, no una cadena obligatoria.

Resultado documentado: aprobado.

Caso 40. Alias de veinte caracteres es válido

Origen: api.feature, línea 800 (../api.feature#L800). **Etiquetas:** `@HU-G1-03`, `@HU-G1-03-CA1`.

Crea una programada con el alias 12345678901234567890 (20 caracteres). Espera 201 y la devolución del mismo alias. Es el límite máximo válido.

Resultado documentado: aprobado.

Caso 41. Alias de veintiún caracteres se rechaza

Origen: api.feature, línea 808 (../api.feature#L808). **Etiquetas:** `@HU-G1-03`, `@HU-G1-03-CA1`.

Intenta crear una programada con 21 caracteres de alias. Espera 422 PROGRAMACION_INVALIDA. Es el primer valor por encima del máximo permitido.

Resultado documentado: aprobado.

Caso 42. Celular inválido al programar

Origen: api.feature, línea 816 (../api.feature#L816). **Etiquetas:** `@HU-G1-03`, `@HU-G1-03-CA1`.

Intenta programar al celular 899888777, que no empieza con 9. Espera 422 PROGRAMACION_INVALIDA. En este endpoint se usa el error de programación, aunque la recarga manual utiliza CELULAR_INVALIDO.

Resultado documentado: aprobado.

Caso 43. Monto no permitido al programar

Origen: api.feature, línea 824 (../api.feature#L824). **Etiquetas:** `@HU-G1-03`, `@HU-G1-03-CA1`.

Intenta programar una recarga de S/ 15. Espera 422 PROGRAMACION_INVALIDA: las programadas también deben respetar los montos permitidos.

Resultado documentado: aprobado.

Caso 44. Próxima ejecución desde 2026-01-10 para día 15

Origen: api.feature, línea 832 (../api.feature#L832). **Etiquetas:** `@HU-G1-03`, `@HU-G1-03-CA2`.

Fija el reloj al 2026-01-10, vuelve a iniciar sesión y crea una programada para el día 15. Espera estado ACTIVA y próxima ejecución `2026-01-15`, porque el día 15 todavía no pasó.

Resultado documentado: aprobado.

Caso 45. Próxima ejecución desde 2026-01-15 para día 15

Origen: api.feature, línea 852 (../api.feature#L852). **Etiquetas:** `@HU-G1-03`, `@HU-G1-03-CA2`.

Fija el reloj al 2026-01-15, vuelve a iniciar sesión y crea una programada para el día 15. Espera estado ACTIVA y próxima ejecución `2026-02-15`, porque hoy ya es día 15 y la próxima ejecución debe ser el mes siguiente.

Resultado documentado: aprobado.

Caso 46. Próxima ejecución desde 2026-01-20 para día 15

Origen: api.feature, línea 872 (../api.feature#L872). **Etiquetas:** `@HU-G1-03`, `@HU-G1-03-CA2`.

Fija el reloj al 2026-01-20, vuelve a iniciar sesión y crea una programada para el día 15. Espera estado ACTIVA y próxima ejecución `2026-02-15`, porque el día 15 ya pasó.

Resultado documentado: aprobado.

Caso 47. Próxima ejecución desde 2026-12-20 para día 15

Origen: api.feature, línea 892 (../api.feature#L892). **Etiquetas:** `@HU-G1-03`, `@HU-G1-03-CA2`.

Fija el reloj al 2026-12-20, vuelve a iniciar sesión y crea una programada para el día 15. Espera estado ACTIVA y próxima ejecución `2027-01-15`, porque el día 15 ya pasó. También comprueba el cambio de año de diciembre a enero.

Resultado documentado: aprobado.

Caso 48. Listado muestra campos y resultado sin ejecuciones

Origen: api.feature, línea 912 (../api.feature#L912). **Etiquetas:** `@HU-G1-04`.

Crea una programada con alias Prueba, monto 10 y día 15. Busca su ID en el listado y exige exactamente una coincidencia. Comprueba celular, alias, monto, día, estado ACTIVA, próxima fecha, fecha de creación y resultado SIN_EJECUCIONES, con detalle y última ejecución nulos. Distingue una programación creada de una que ya fue procesada.

Resultado documentado: aprobado.

Caso 49. Editar monto alias y día recalcula fecha sin cambiar el celular

Origen: api.feature, línea 927 (../api.feature#L927). **Etiquetas:** `@HU-G1-05`, `@HU-G1-05-CA1`.

Fija el reloj al 10 de enero de 2026 y crea una programada para el día 15. Mediante PUT cambia monto a 20, alias a Nuevo y día a 8. Espera próxima ejecución 2026-02-08, ya que el día 8 de enero pasó. El celular debe mantenerse. Vuelve a consultar para verificar que el nuevo alias quedó guardado.

Resultado documentado: aprobado.

Caso 50. Edición inválida no modifica la programada { monto: 15 }

Origen: api.feature, línea 957 (../api.feature#L957). **Etiquetas:** `@HU-G1-05`, `@HU-G1-05-CA1`.

Crea una programada válida y guarda su respuesta completa. Envía PUT con `{ monto: 15 }`. Espera 422 PROGRAMACION_INVALIDA; al volver a listar, la programada debe ser idéntica a la original. Verifica el rechazo y que la edición fallida no deje cambios parciales.

Resultado documentado: aprobado.

Caso 51. Edición inválida no modifica la programada { dia: 29 }

Origen: api.feature, línea 978 (../api.feature#L978). **Etiquetas:** `@HU-G1-05`, `@HU-G1-05-CA1`.

Crea una programada válida y guarda su respuesta completa. Envía PUT con `{ dia: 29 }`. Espera 422 PROGRAMACION_INVALIDA; al volver a listar, la programada debe ser idéntica a la original. Verifica el rechazo y que la edición fallida no deje cambios parciales.

Resultado documentado: aprobado.

Caso 52. Edición inválida no modifica la programada { alias: '123456789012345678901' }

Origen: api.feature, línea 999 (../api.feature#L999). **Etiquetas:** `@HU-G1-05`, `@HU-G1-05-CA1`.

Crea una programada válida y guarda su respuesta completa. Envía PUT con `{ alias: '123456789012345678901' }`. Espera 422 PROGRAMACION_INVALIDA; al volver a listar, la programada debe ser idéntica a la original. Verifica el rechazo y que la edición fallida no deje cambios parciales.

Resultado documentado: aprobado.

Caso 53. Rechazar transición inválida pausar

Origen: api.feature, línea 1020 (../api.feature#L1020). **Etiquetas:** `@HU-G1-05`, `@HU-G1-05-CA3`.

Crea una programada activa y la pausa correctamente con 200. Intenta pausarla otra vez: espera 409 ESTADO_INVALIDO. Un estado ya pausado no admite repetir esa transición.

Resultado documentado: aprobado.

Caso 54. Rechazar transición inválida reanudar

Origen: api.feature, línea 1037 (../api.feature#L1037). **Etiquetas:** `@HU-G1-05`, `@HU-G1-05-CA3`.

Crea una programada ACTIVA e intenta reanudarla directamente. Espera 409 ESTADO_INVALIDO, porque reanudar solo corresponde a una PAUSADA.

Resultado documentado: aprobado.

Caso 55. Pausada no se ejecuta y reanudar recalcula desde hoy

Origen: api.feature, línea 1050 (../api.feature#L1050). **Etiquetas:** `@HU-G1-05`, `@HU-G1-05-CA2`.

Aísla el proceso diario y fija el reloj al 10 de enero. Crea una programada para el 15 y la pausa. Avanza al 16, inicia sesión y procesa: saldo 1500.00 y resultado SIN_EJECUCIONES. Al reanudar espera estado ACTIVA y próxima fecha 2026-02-15, calculada desde el día actual, no desde la antigua fecha vencida.

Resultado documentado: aprobado.

Caso 56. Eliminar oculta la programada e impide futuras ejecuciones conservando historial

Origen: api.feature, línea 1122 (../api.feature#L1122). **Etiquetas:** `@HU-G1-06`.

Con datos del proceso aislados, crea una programada para el 15 y la ejecuta ese día. Guarda su única recarga histórica. Elimina la programada con 204 y verifica que desaparezca del listado. Avanza al vencimiento del mes siguiente, vuelve a procesar y confirma que el historial permanezca igual. Eliminar una programación no borra dinero ya operado ni permite nuevas ejecuciones.

Resultado documentado: aprobado.

Caso 57. No ejecutar programada antes de su fecha

Origen: api.feature, línea 1205 (../api.feature#L1205). **Etiquetas:** `@HU-G1-07`, `@HU-G1-07-CA1`.

Crea una programada para el 15 de enero y ejecuta el proceso el día 14. Espera cero recargas exitosas, saldo 1500.00 y resultado SIN_EJECUCIONES. Impide ejecutar una operación antes de su vencimiento.

Resultado documentado: aprobado.

Caso 58. Programada por S/ 30 aplica bono y registra origen e historial

Origen: api.feature, línea 1269 (../api.feature#L1269). **Etiquetas:** `@HU-G1-07`, `@HU-G1-07-CA2`.

Aísla el proceso diario y fija el reloj al 10 de enero. Crea una programada de S/ 30 para el 15 y procesa en esa fecha. En el listado espera EXITOSA, detalle nulo, fecha de ejecución textual y próxima fecha

2026-02-15. En el historial del destino debe existir una única operación con monto 30.00, bono 1.50 y origen PROGRAMADA. El caso verifica el bono automático; no contiene una consulta específica del saldo final.

Resultado documentado: fallido. DEF-03: devuelve bono 0.00 en lugar de 1.50. Los pasos posteriores al fallo no llegaron a verificarse.

Caso 59. Programada por S/ 50 aplica bono y registra origen e historial

Origen: api.feature, línea 1334 (../api.feature#L1334). **Etiquetas:** `@HU-G1-07`, `@HU-G1-07-CA2`.

Aísla el proceso diario y fija el reloj al 10 de enero. Crea una programada de S/ 50 para el 15 y procesa en esa fecha. En el listado espera EXITOSA, detalle nulo, fecha de ejecución textual y próxima fecha 2026-02-15. En el historial del destino debe existir una única operación con monto 50.00, bono 2.50 y origen PROGRAMADA. El caso verifica el bono automático; no contiene una consulta específica del saldo final.

Resultado documentado: aprobado.

Caso 60. Programada por S/ 100 aplica bono y registra origen e historial

Origen: api.feature, línea 1399 (../api.feature#L1399). **Etiquetas:** `@HU-G1-07`, `@HU-G1-07-CA2`.

Aísla el proceso diario y fija el reloj al 10 de enero. Crea una programada de S/ 100 para el 15 y procesa en esa fecha. En el listado espera EXITOSA, detalle nulo, fecha de ejecución textual y próxima fecha 2026-02-15. En el historial del destino debe existir una única operación con monto 100.00, bono 4.00 y origen PROGRAMADA. El caso verifica el bono automático; no contiene una consulta específica del saldo final.

Resultado documentado: aprobado.

Caso 61. Fallo por saldo insuficiente conserva saldo informa SMS y avanza fecha

Origen: api.feature, línea 1464 (../api.feature#L1464). **Etiquetas:** `@HU-G1-07`, `@HU-G1-07-CA3`, `@HU-G1-07-CA4`.

Aísla el proceso, crea una programada de S/ 10 y fija saldo 9.99. El día 15 ejecuta el proceso: debe registrar una recarga fallida, conservar saldo 9.99 y no agregar una recarga al historial del destino. En el listado espera FALLIDA, detalle textual y próxima ejecución 2026-02-15. El titular debe recibir un SMS que contenga Saldo insuficiente. La falta de saldo no deja la misma ejecución pendiente indefinidamente.

Resultado documentado: aprobado.

Caso 62. Varios meses atrasados se ejecutan una sola vez y avanzan al próximo mes

Origen: api.feature, línea 1542 (../api.feature#L1542). **Etiquetas:** `@HU-G1-07`, `@HU-G1-07-CA4`.

Crea el 10 de enero una programada para el 15 y avanza directamente al 20 de abril. El proceso debe hacer una sola recarga de S/ 10, dejar saldo 1490.00 y próxima ejecución 2026-05-15. No debe cobrar una operación por cada mes acumulado.

Resultado documentado: aprobado.

Caso 63. Procesar dos veces el mismo día no duplica recargas

Origen: api.feature, línea 1606 (../api.feature#L1606). **Etiquetas:** `@HU-G1-07`, `@HU-G1-07-CA5`.

Crea una programada para el 15 y ejecuta el proceso en esa fecha. Guarda el historial. Vuelve a ejecutar el proceso sin avanzar el reloj: debe informar cero recargas exitosas, devolver el mismo historial y conservar saldo 1490.00. Verifica idempotencia: repetir la orden no repite el cobro.

Resultado documentado: aprobado.

Caso 64. Sin sesión no se permite get 'api/recargas'

Origen: api.feature, línea 1675 (../api.feature#L1675). **Etiquetas:** `@seguridad-recargas`.

Después de la preparación común elimina el header Authorization. Ejecuta GET sobre 'api/recargas'. Espera 401 NO_AUTENTICADO. Aunque el Background obtuvo una sesión, esta solicitud no envía su token y debe rechazarse.

Resultado documentado: aprobado.

Caso 65. Sin sesión no se permite post 'api/recargas'

Origen: api.feature, línea 1683 (../api.feature#L1683). **Etiquetas:** `@seguridad-recargas`.

Después de la preparación común elimina el header Authorization. Ejecuta POST sobre 'api/recargas'. Espera 401 NO_AUTENTICADO. Aunque el Background obtuvo una sesión, esta solicitud no envía su token y debe rechazarse.

Resultado documentado: aprobado.

Caso 66. Sin sesión no se permite get 'api/recargas/programadas'

Origen: api.feature, línea 1692 (../api.feature#L1692). **Etiquetas:** `@seguridad-recargas`.

Después de la preparación común elimina el header Authorization. Ejecuta GET sobre 'api/recargas/programadas'. Espera 401 NO_AUTENTICADO. Aunque el Background obtuvo una sesión, esta solicitud no envía su token y debe rechazarse.

Resultado documentado: aprobado.

Caso 67. Sin sesión no se permite post 'api/recargas/programadas'

Origen: api.feature, línea 1700 (../api.feature#L1700). **Etiquetas:** `@seguridad-recargas`.

Después de la preparación común elimina el header Authorization. Ejecuta POST sobre 'api/recargas/programadas'. Espera 401 NO_AUTENTICADO. Aunque el Background obtuvo una sesión, esta solicitud no envía su token y debe rechazarse.

Resultado documentado: aprobado.

Caso 68. Otro usuario no puede editar una programada de Ana

Origen: api.feature, línea 1709 (../api.feature#L1709). **Etiquetas:** `@seguridad-recargas`.

Crea una programada como Ana y guarda su ID. Inicia sesión como Bruno (912345678, PIN 7391), cambia el header a su token e intenta editar la programada de Ana. Espera 404 PROGRAMADA_NO_ENCONTRADA. El ID válido no concede acceso a un recurso ajeno y la API debe

responder como si no existiera.

Resultado documentado: aprobado.

Caso 69. Otro usuario no puede pausar una programada de Ana

Origen: api.feature, línea 1728 (../api.feature#L1728). **Etiquetas:** `@seguridad-recargas`.

Crea una programada como Ana y guarda su ID. Inicia sesión como Bruno (912345678, PIN 7391), cambia el header a su token e intenta pausar la programada de Ana. Espera 404 PROGRAMADA_NO_ENCONTRADA. El ID válido no concede acceso a un recurso ajeno y la API debe responder como si no existiera.

Resultado documentado: aprobado.

Caso 70. Otro usuario no puede reanudar una programada de Ana

Origen: api.feature, línea 1747 (../api.feature#L1747). **Etiquetas:** `@seguridad-recargas`.

Crea una programada como Ana y guarda su ID. Inicia sesión como Bruno (912345678, PIN 7391), cambia el header a su token e intenta reanudar la programada de Ana. Espera 404 PROGRAMADA_NO_ENCONTRADA. El ID válido no concede acceso a un recurso ajeno y la API debe responder como si no existiera.

Resultado documentado: aprobado.

Caso 71. Otro usuario no puede eliminar una programada de Ana

Origen: api.feature, línea 1766 (../api.feature#L1766). **Etiquetas:** `@seguridad-recargas`.

Crea una programada como Ana y guarda su ID. Inicia sesión como Bruno (912345678, PIN 7391), cambia el header a su token e intenta eliminar la programada de Ana. Espera 404 PROGRAMADA_NO_ENCONTRADA. El ID válido no concede acceso a un recurso ajeno y la API debe responder como si no existiera.

Resultado documentado: aprobado.

Ejecutar y contrastar con el reporte

```
source env/bin/activate
python scripts/karate_local.py --tags @recargas
```

Para encontrar un caso por nombre:

```
python scripts/karate_local.py --tags @recargas --name 'Saldo exacto'
```

El número «Caso 01», etc., organiza este documento; no es una etiqueta ejecutable. Para seleccionar pruebas se usan etiquetas o nombres del archivo feature.

Abre `reportes/karate/karate-summary.html`, entra en `api.feature` y busca el nombre del caso. La primera aserción roja explica la diferencia entre lo esperado y lo obtenido. El detalle de los tres defectos está en defectos.md (defectos.md).

La prueba de interfaz de HU-G1-01 CA4 sigue pendiente: consultar la configuración no demuestra que la pantalla muestre el bono antes de confirmar. Tampoco se están probando todas las combinaciones posibles de usuarios, fechas y montos.